

the makefiles. You can see this in the following screenshot.

Makefiles are instruction for the **make** program, which is nothing but a utility for automatically building executable programs and libraries from source code. The makefiles actually specify how to derive the target program from each of its dependencies.

In simple terms, the makefile associated with the “hello world” source code written in C++ instructs the “make” program to call the relevant compiler (g++) and that in order to compile this software, it needs the “Abra” dependency located in the “Ca Dabra” location.

Now the obvious question arises that how does the developer of the software will know where is “Ca Dabra” location on each computer. So there must be a method to dynamically generate the makefiles by examining the system and ensuring all relevant dependencies and compilers are installed and at what location. Well that is the function of the “Configure” Script above.

The configure script, when executed checks for environmental variables, install locations, necessary dependencies, tools and other stuff and if everything that is needed is there, passes on this information to the makefiles, which in turn tell the make program what to do.

The other file, “digit.cpp” in the “src” folder simply contains

```
21
22
23
24     #include <iostream>
25
26     using namespace std;
27
28     int main(int argc, char *argv[])
29     {
30         cout << "Hello Readers of Digit!" << endl;
31
32         return EXIT_SUCCESS;
33     }
```

Some of the code in the digit.cpp file

the actual source, which is as follows.

As you can see you can easily modify this source code file to suit your requirements. Similarly if you know the language in which the software that you are compiling is written, you can